Architecture

The description of the architecture of the system.

The diagrams are built using PlantUML with some diagrams utilizing the C4 model extension available here: https://github.com/plantuml-stdlib/C4-PlantUML#deployment-diagram

If and when changes are made to the diagrams (and when the architecture changes), they can be recreated under `docs/architecture/imgs` with:

```
[project_root]$ cd docs/architecture/
[architecture]$ plantuml diag_src/*.puml -o ../imgs
```

NOTE You need to have 'plantuml' installed, the C4 model extension should be fetched automatically from the github repository.

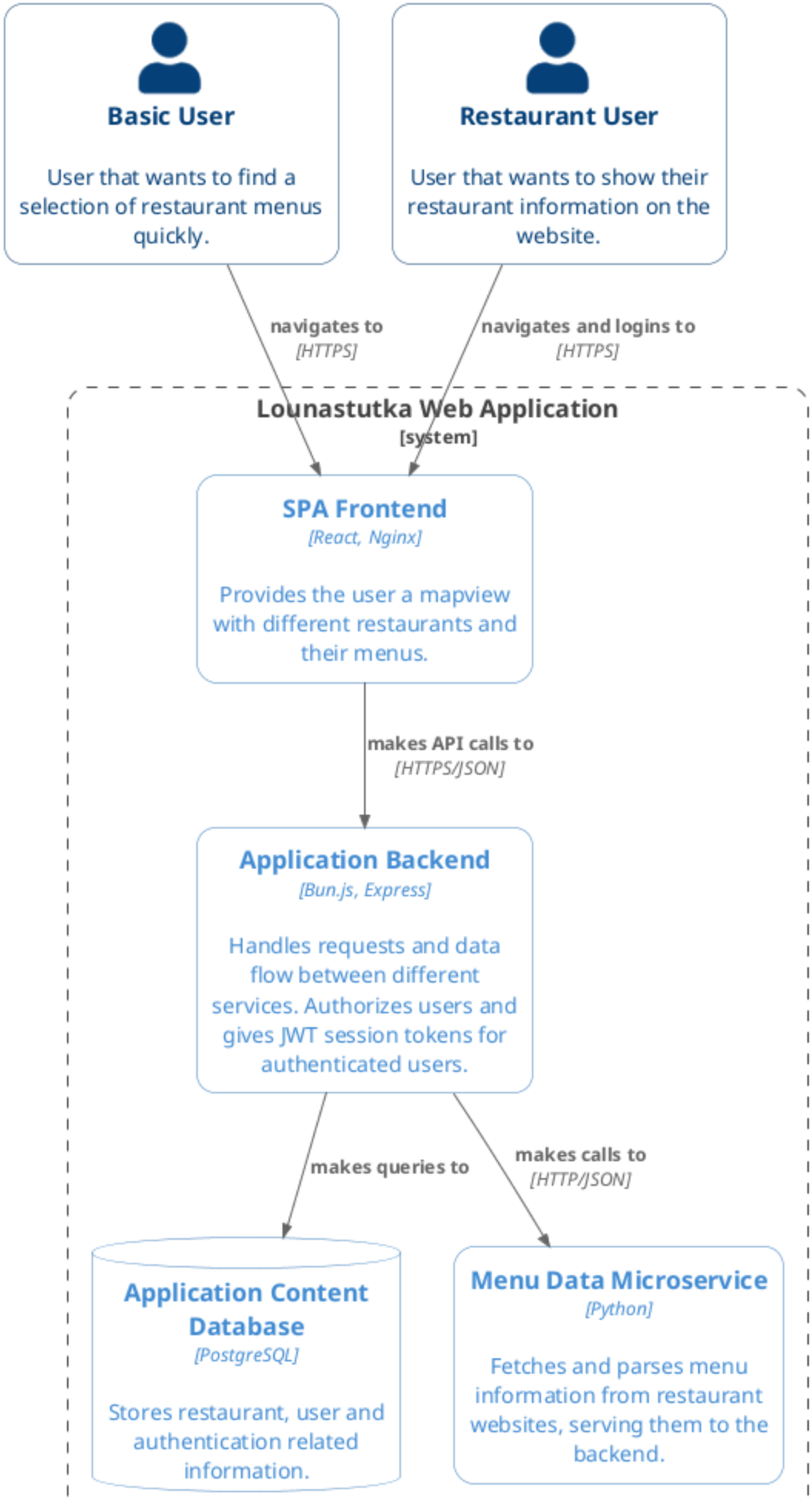
The Architecture

The main application components are described here, without the infrastructure components such as traefik, grafana, portainer, docker etc. as they are shown within the deployment diagram. The diagram style utilized here is the C4 models "Container" view and is built with `plantuml`.

Containers:

- **Basic User:** User that does not necessarly want to login but wants to check local restaurant menus via interactive map.
- **Restaurant User:** User who might want to add their restaurant information on the website for other users to see.
- **Frontend (bunjs nginx react):** Responsible for showing user the main content of the application. This is the service user directly interacts with, others are internal and only accessible through APIs.
- **Backend (bunjs express):** Handles the data flow between frontend, microservice and database.
- **Microservice (python):** Scrapes a website based on the given URL and tries to gather menu data.
- **Database (postgresql):** Stores applications content such as information about the restaurants as well as metadata about the users that want to authenticate.

Main Component Architecture - Lounastutka System

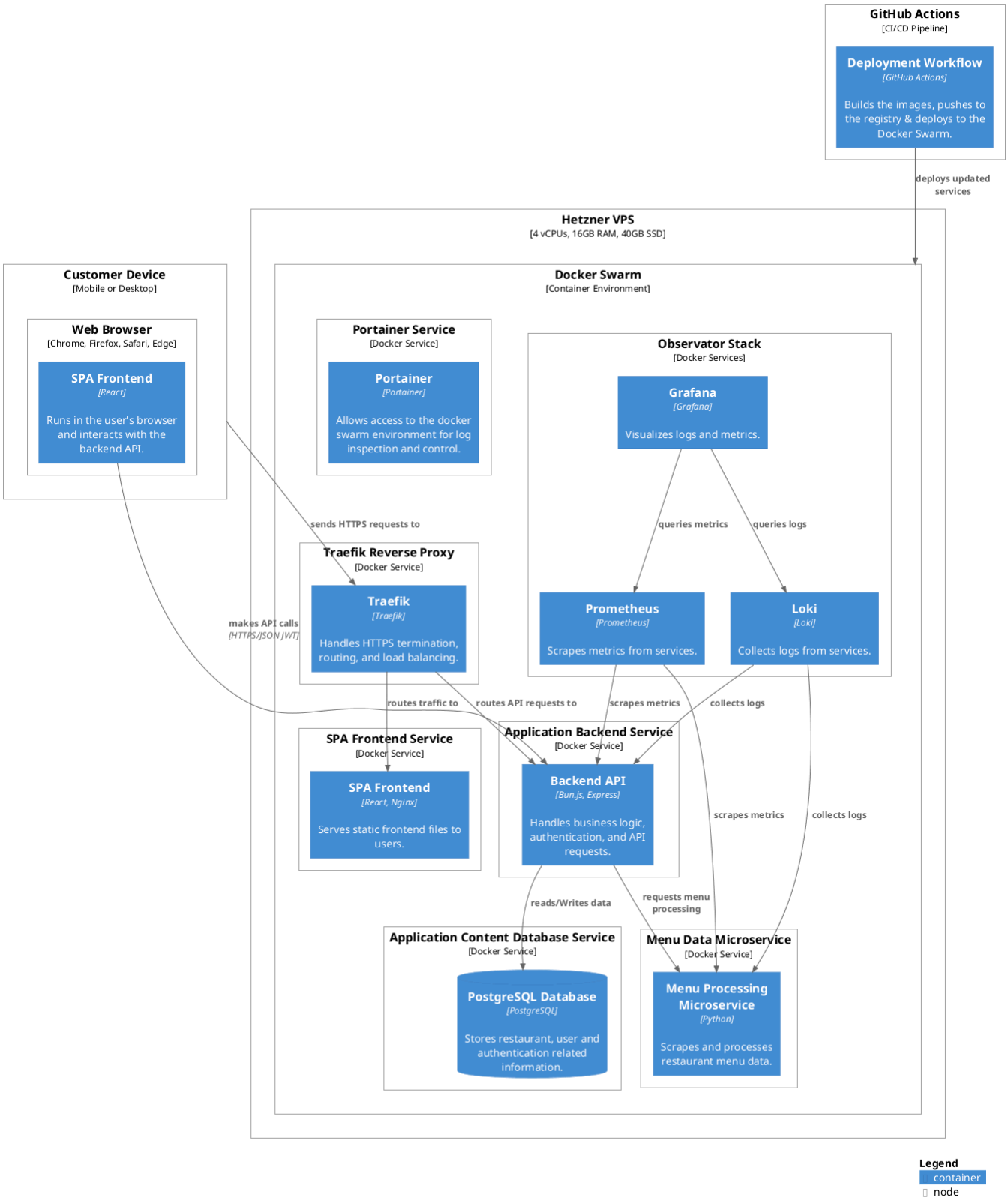


Legend
person
container
system boundary

Deployment diagram

The system with infrastructure services such as Traefik, Grafana, Loki and prometheus frameworks. The internal services are all deployed on a single VPS at Hetzner server farm. The github actions is external system and connects to the internal system via CI/CD pipeline.

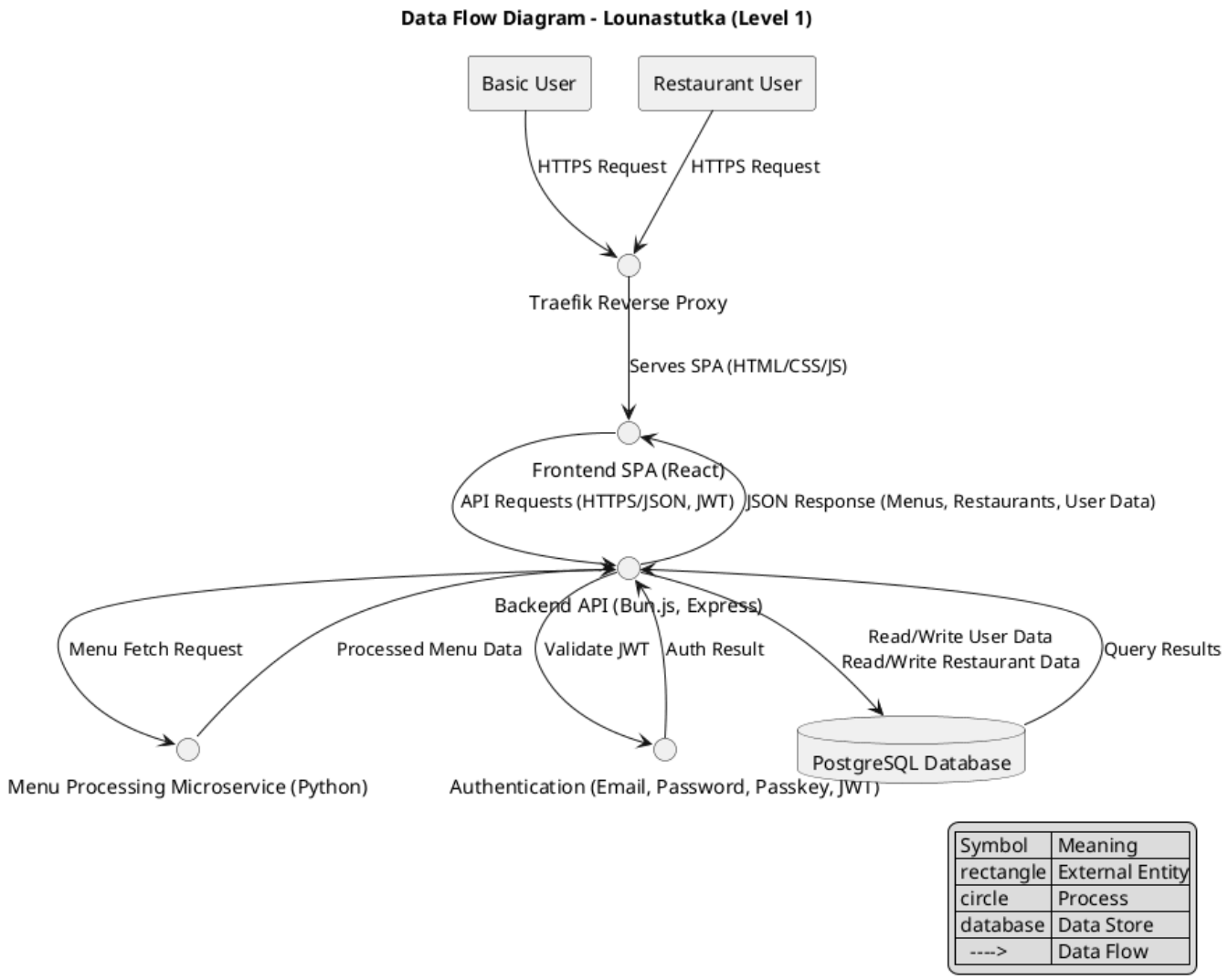
Deployment Diagram - Lounastutka System



Data flow diagram

The diagram shows the Data Flow Diagram Level 1 view with the "Yourdon and Coad"-style as descriped in lucidcharts website here: https://www.lucidchart.com/pages/data-flow-diagram

Most of the data flows are bi-directional, and the users interaction with traefik are described only as one-directional due to the initial navigation to the website (SPA frontend) from which the data flows between the software system. Circles are the processes between which data flows in the manner as the arrows describe. Rectangles are the external entities such as users, the github actions could in theory be described here as it is external entity from which data is flowing to the system when updates happen but it is shown in the deployment diagram and felt out of place here. The database is the only data store element within the system.

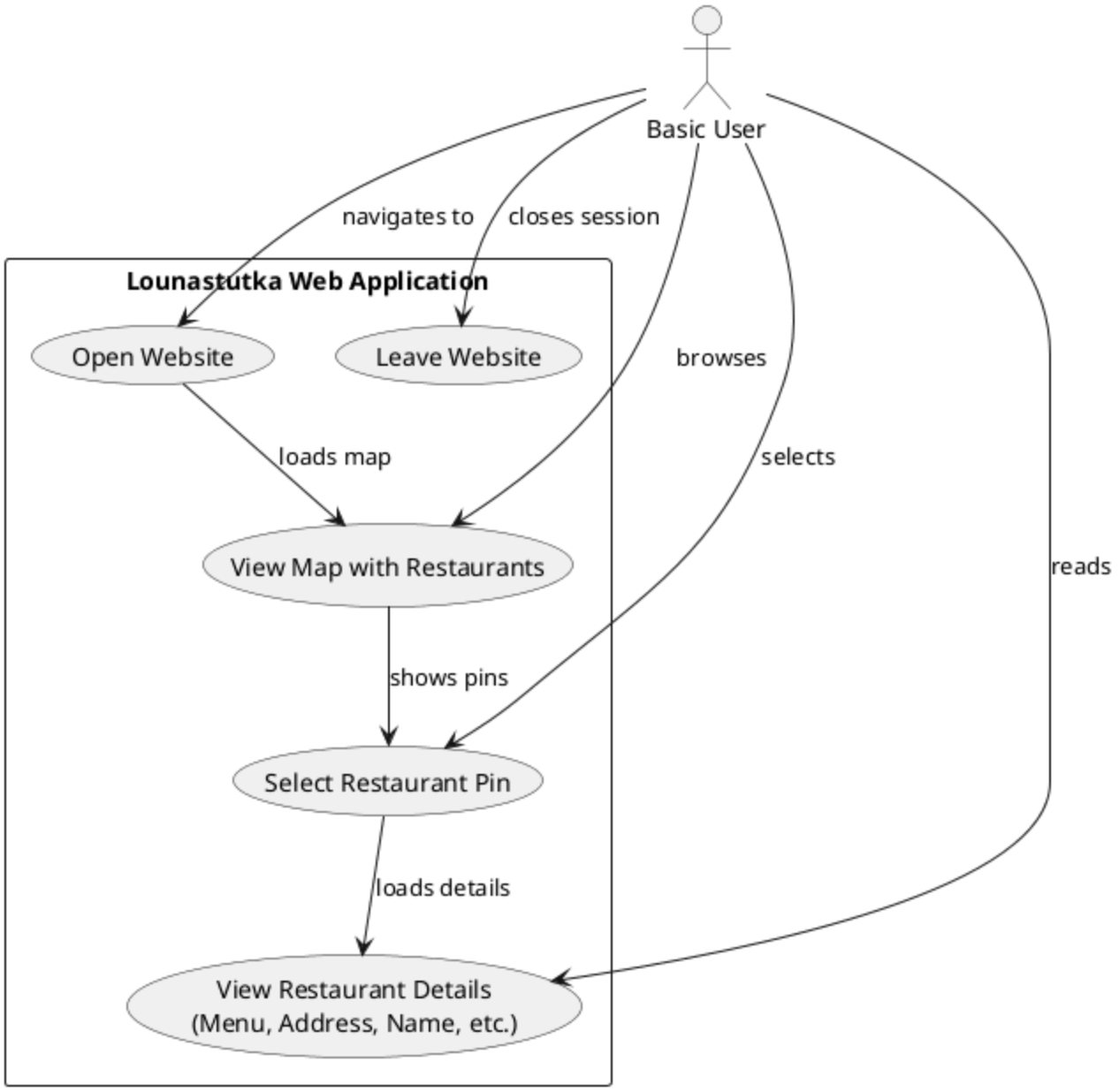


Use-case diagrams

The application has two distinct use-cases, the basic user use-case where a user wants to just see whats on the menu today on local restaurants and then close the site.

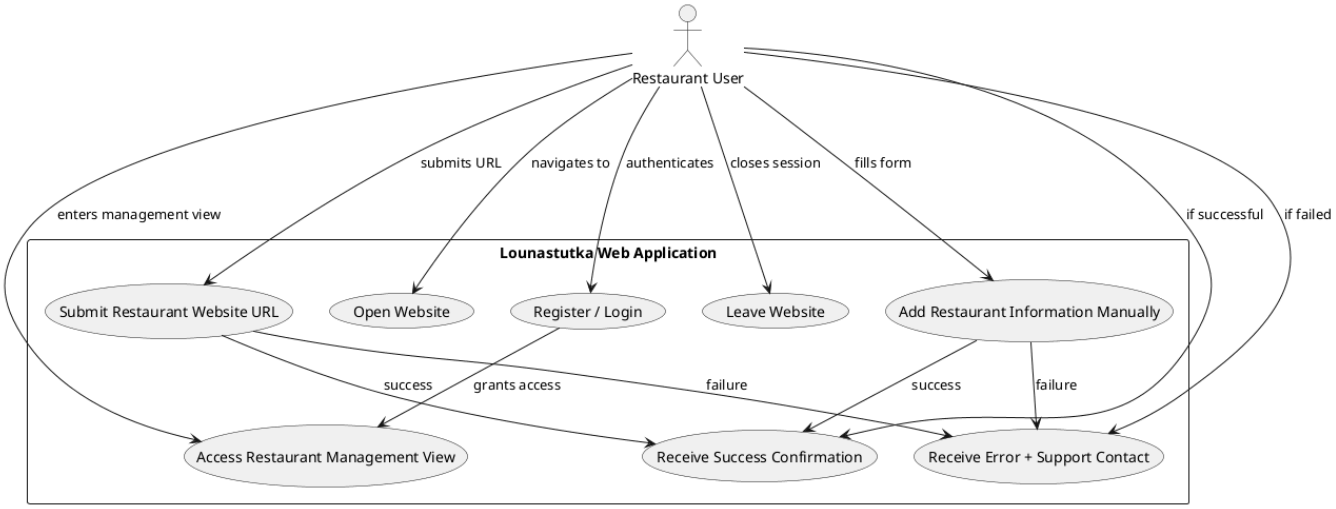
The other more complex use-case is for the restaurant users who might want to add their restaurant information to the website to be seen by the normal users, by either manually typing the information or providing an URL that is used to fetch the information from the restaurants website. As the addition might fail due to errors in data transfer, the user is given some additional information about the status.

Use Case - Basic User Browsing Restaurant Menus



Symbol	Meaning
Rectangle	Boundary
Stickman	User
Ellipse	Use-case
Arrow	Action

Use Case - Restaurant User Adding Restaurant Information



Symbol	Meaning
Rectangle	Boundary
Stickman	User
Ellipse	Use-case
Arrow	Action